B.Tech Project Progress Report

Academic Year 2025–26

LLMs for Industrial Co-pilots and Dashboards

LLM-Augmented Pipelines for Telecom Network Fault Diagnosis and Autonomous KPI Analysis

---

Team Members	Akshat Gupta(12340160) Rajeev Kumar(12341700) Rishi Kharya(12341790)
Mentor	Chaoub Abdelaali
Supervisor	Dr. Gagan Raj Gupta

Project Links

GitHub: github

Research Paper: View Paper

Department of Computer Science and Engineering

April 2026

Contents

Introduction

This report documents B.Tech project work carried out from January to April 2026 across two parallel workstreams. For the first, we participated in the AI Telco Troubleshooting Challenge, a Zindi competition focused on automated root-cause analysis of 5G network faults from drive-test logs. The second is a set of contributions to the Trace AI Agent, a system that allows network operations staff to query KPI data through a conversational interface.

The main idea for both tracks is how to combine deterministic computation with LLM reasoning, taking advantage of each's strengths. Over four months, this question led to a shift from a simple prompt-engineering pipeline to a complete tool-based agentic system with retrieval-augmented memory. The performance of the lightweight Qwen2.5-1.5B-Instruct model on the competition's test set increased from 65

AI Telco Challenge

Problem and Constraints

The AI Telco Troubleshooting Challenge[1] provides raw telecom telelogs and asks participants to identify the root cause of a network fault from a set of labeled options (C1 through C8). Three model tiers are evaluated: Qwen3-32B, Qwen2.5-7B-Instruct, and Qwen2.5-1.5B-Instruct.

The test set follows a two-phase release schedule. Phase 1 questions were available from 28 November to 17 January; Phase 2 questions were released on 19 January. This structure was specifically designed to prevent overfitting to the test distribution, so the pipeline had to generalise across unseen fault types and data layouts from the start.

Pipeline Design

The core system is a four-stage sequential pipeline. Each stage handles one concern: parsing, feature extraction, reasoning preparation, and answer extraction.

Stage 1: Dynamic Data Extraction

Raw questions contain pipe-delimited tables in two distinct formats. Format 1 (Phase 1 questions) places user-plane data before engineering parameters and uses C1–C8 option labels directly. Format 2 (Phase 2) reverses the section order and uses numeric options (1–8) that must be mapped back to C-labels via keyword matching.

Several practical complications arose during implementation:

- Column headers in the dataset are highly inconsistent. The same field appears as "gps speed", "GPS Speed", "GPS Speed [km/h]" and other variants depending on the question. We built a column alias registry with substring-based, case-insensitive matching to handle this without hardcoding every variant.
- The dataset contains a consistent typo in section markers ("Engeneering" instead of "Engineering") that the parser has to account for.

[1] https://zindi.africa/competitions/the-ai-telco-troubleshooting-challenge

- For Phase 2 questions, the numeric option labels (e.g., option "3") must be resolved to canonical C-labels by matching option text against a keyword dictionary – for instance, "downtilt" maps to C1, "overshooting" maps to C2.
- Rows are filtered to a "drop period," which includes rows where throughput is below a set threshold (default 600 Mbps). Subsequent calculations only use this filtered subset, representing the actual fault window.

The extractor produces a structured object with optional fields. Each field is `None` if the related data was not found. This setup flows smoothly through later stages without causing crashes.

Stage 2: Feature Engineering

Eleven diagnostic features are computed from the extracted data. The focus is on measurements that directly correspond to the eight root-cause hypotheses:

- **Mobility indicators:** Maximum GPS speed, average and minimum resource block count, handover event count (derived from PCI transitions in the drop period).
- **Geometry features:** Per-row distance from serving cell computed via the Haversine formula; the feature records the *maximum* per-row distance rather than the mean. This was a deliberate correction from earlier versions – C1 (excessive downtilt) manifests as throughput collapse at moderate distances (0.1–0.4 km), which a mean distance calculation was washing out.
- **Interference indicators:** Serving PCI modulo 30, strongest neighbour PCI modulo 30, count of all neighbours whose PCI modulo 30 matches the serving cell. The modulo 30 check is how LTE/5G PCI collision is formally defined – cells sharing the same mod-30 value interfere in the synchronisation channel.
- **Antenna configuration:** Mechanical downtilt, digital tilt, total downtilt (sum), vertical beamwidth, and a downtilt ratio (total downtilt / vertical beamwidth). The ratio is a compound feature that turned out to be the most discriminating for C1.
- **Colocation flag:** Boolean derived by comparing the serving cell's gNodeB ID with the strongest neighbour's gNodeB ID. If they share an ID, the neighbour is colocated (i.e., on the same physical site), which changes the interpretation of RSRP and interference readings.

Stage 3: Dual-Mode Classification

Two reasoning strategies were implemented, selectable per sample:

Elimination Mode uses a fixed priority hierarchy to check each root cause in sequence and eliminate infeasible ones early. The priority order – C7 > C8 > C5 > C6 > C2 > C4 > C1 > C3 – was established empirically from dataset analysis. Two corrections from the naive ordering were important: C5 (ping-pong handovers) must be checked before C6 (PCI mod-30 collision) because C6 is often a symptom of the same event, and C2 (overshooting) must be checked before C4 (non-colocated interference) because strong neighbour RSRP at large distances is geometrically expected and should not be classified as interference.

Thorough Mode evaluates all eight root causes in parallel using a three-tier status system: FEASIBLE (hard constraint satisfied), UNLIKELY (borderline), and ELIMINATED

(physical constraint violated). Each cause gets a confidence score, and a conflict-resolution step applies priority ranking when multiple causes are feasible. C3 (better neighbour available) is always marked FEASIBLE as a catch-all fallback, ensuring at least one valid option is always present.

Inference time is approximately 1.5 s per sample in Elimination Mode and 2–3 s in Thorough Mode.

Stage 4: Answer Extraction and Validation

LLM output format is not reliably consistent. We applied a seven-pattern cascading extraction strategy:

1. Regex match for $\boxed{\text{CX}}$ (canonical competition format).
2. Regex match for $\boxed{N}$ (numeric, Phase 2 only; converted via option map).
3. Phrase match for "Answer:", "Verdict:", or "Conclusion:" prefix.
4. Bold markdown match (`**CX**`).
5. Sentence pattern match ("root cause is CX", "cause = CX").
6. Last standalone `\b(C[1-8])\b` token in the full response.
7. Last standalone digit 1–8 (Phase 2 only, converted via option map).

After extraction, if the predicted label falls in the eliminated causes list from Stage 3, it is overridden with the top-priority feasible cause. Any label not present in the actual option set for that question is rejected and the next extraction method is tried. This multi-layer strategy made a measurable difference for the 1.5B model, which produced far more format inconsistencies than the larger models.

Multi-Run Ensemble

Each question is processed four times independently, and the final prediction is determined by majority vote across the four runs. This improved robustness against the sampling variance of local LLM inference. Such processing is particularly important for the 1.5B model where individual run outputs vary noticeably even with the same input.

Result

Leaderboard rank	23 / 199
Qwen2.5-1.5B-Instruct accuracy	65%

Figure 1: Rank

Trace AI Agent – Feature Extensions

Background

The Trace AI Agent is a conversational system that allows network operations staff to query CDR (Call Detail Record) data by describing what they want in plain English. The core workflow is built on LangGraph and translates user prompts into structured filter expressions over a CDR schema, executes them, and optionally generates visualisations. Work in February focused on three additions to this system: a semantic caching layer, an autonomous error correction subsystem, and prompt refinement.

Intelligent Caching

KPI generation involves multiple LLM calls and is expensive to repeat for similar queries. We planned to implement a three-level cache hierarchy:

- **Level 1 – Exact match:** Normalised KPI name (lowercased, whitespace-collapsed) is looked up directly in a JSON-backed dictionary. This is the fast path and handles repeated identical queries.
- **Level 2 – Semantic similarity:** If no exact match exists, the query is embedded using Azure OpenAI's `text-embedding-3-small` model and compared against cached embeddings via cosine similarity. A configurable threshold (default 0.85) determines whether a cached result is close enough to reuse. Embeddings are themselves cached by MD5 hash of the input text to avoid redundant API calls.
- **Level 3 – Query pattern:** The full natural-language query is hashed and checked against a pattern cache. This handles cases where the same underlying intent is expressed with slightly different phrasing.

Error Taxonomy for Autonomous Correction

Filters fail in consistent, identifiable ways. We catalogued these into five categories:

- **Syntax errors (5 types):** Invalid comparison operators, missing or mismatched quotes, unbalanced parentheses, invalid wildcard usage.
- **Schema errors (4 types):** Field names not present in the CDR schema, missing required prefixes (e.g., `cdr.`), incorrect or missing enrichment suffixes.
- **Semantic errors (4 types):** Type mismatches between field type and comparison value, mixing protocols in a single filter expression (e.g., SIP and GTP fields in one condition), logical contradictions (e.g., a field simultaneously equals two incompatible values), invalid threshold values.
- **Data type errors (3 types):** String values compared without quotes, numeric values wrapped in quotes, boolean field errors.
- **Domain-specific errors (5 types):** SIP response codes outside valid range (100–699), wrong response field names for a given protocol, missing config placeholders, incomplete filter conditions.

Prompt Refinements

The chatbot and KPI agent system prompts were also revised using a set of previously failing queries as negative examples, narrowing the space of problematic output formats. The prompts became more detailed and with more examples, enabling the agent to understand better.

Telco Challenge Pipeline Redesign and Starting with the Paper

Motivation

After the competition phase ended, we began treating the pipeline not as a submission system but as the design object for a research paper. The January pipeline had worked well empirically, but its four stages did not have a principled account of why each stage was there or what class of problem it was solving. Reading recent work on hybrid LLM-classical architectures clarified the framing: LLMs are unreliable on arithmetic, formula evaluation, and threshold comparison, but genuinely useful for causal reasoning once the evidence is well-structured and the decision space is constrained. A coherent pipeline design should make this division of labour explicit.

Revised Seven-Stage Architecture

The pipeline was restructured into seven stages with clearly defined roles:

1. **Raw Logs:** Unprocessed telelog input. No assumptions about ordering or format.
2. **Deterministic Computation:** All numeric computation that the LLM would handle inconsistently is done here and only here. This includes:
   - Aggregations – mean, variance, minimum and maximum over the drop period.
   - Temporal metrics – event rates, propagation delays, handover frequency.
   - Event correlations – A5 measurement report counting.
   - Domain formulas – SINR, PER, path loss, Haversine distance.

   The outputs are exact numbers, not approximate LLM estimates. This is the stage where the January pipeline's max-distance correction and the mod-30 collision logic live.

3. **Decision Space Shaping:** A modular control layer that checks whether each possible root cause is physically feasible. It works by eliminating clearly impossible options, verifying feasibility, and applying basic constraints to reduce the set of candidates. Importantly, this stage does not choose the final answer but only removes options that are clearly not possible (for example, C7 cannot be the cause if the maximum speed is below 10 km/h) and marks some others as unlikely. The final decision is still made entirely by the LLM. This is what makes the pipeline different from a rule-based system: the rules only narrow down the choices, they do not decide the answer.
4. **Semantic Abstraction:** Numeric outputs and the reduced candidate set are converted into structured natural-language descriptions that can be easily understood by LLM for reasoning. Rather than presenting raw numbers, this stage produces annotated summaries. Early experiments confirmed that the framing of evidence at this stage has a measurable effect on reasoning quality.
5. **LLM Reasoning:** The model receives the semantic context and is asked to perform causal inference – why do these indicators co-occur, which root cause is most consistent with the full evidence set. This is the only stage where the LLM is involved, and it is limited to reasoning, not computation.
6. **Decision / Action:** The LLM output is extracted and normalised. The same cascading extraction strategy from January is applied here.
7. **Feedback:** Evaluated predictions and pipeline records (features, decision space, LLM response, extraction method, correction flags) are logged to CSV for analysis. The feedback stage also computes per-class accuracy, confusion matrices, and a fallback rate metric. When a class accuracy falls below 50%, it flags the decision shaping logic for that cause for review. When the fallback rate exceeds 10%, it flags the output format instruction in the system prompt.

Result

Qwen2.5-1.5B-Instruct accuracy	72%
Improvement over January	+7 percentage points

The gain came primarily from the semantic abstraction stage. Once evidence was presented in annotated form with explicit threshold comparisons, the 1.5B model's reasoning quality improved noticeably on classes C1, C4, and C6, which are the three most easily confused by a small model working from raw numbers.

Memory and Tool-Based Agent and Paper Finalisation

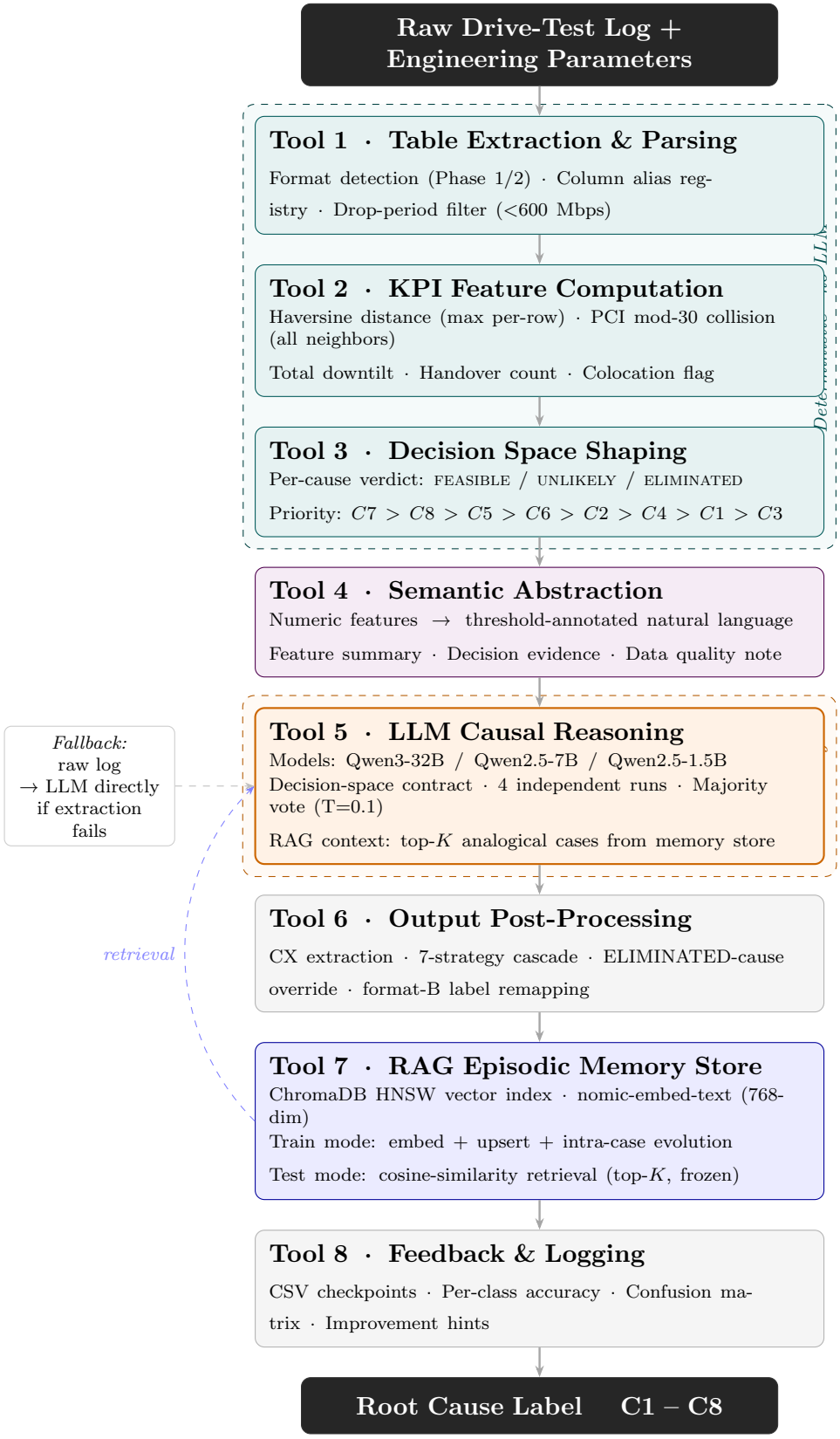
From Pipeline to Agent

The previous pipeline was a fixed sequential chain – every sample went through all seven stages in the same order. In April we restructured the system so that each stage is exposed as an independently callable tool within a ReAct-style agent loop. The agent orchestrator selects and sequences tools based on the structure of each incoming sample. The tools exposed to the agent are:

- **Extract and Compute:** Calls Stages 1 and 2. Parses the drive-test and engineering tables, handles both question formats, maps Phase 2 numeric options to C-labels, and computes the full 52-element feature vector. Reports an `extraction_failed` flag if

key metrics could not be found, which causes the agent to route directly to prediction with a fallback system prompt rather than attempting shaping on incomplete data.

- **Assess Causes:** Calls Stage 3. Evaluates all eight root causes against the computed features and returns a `DecisionSpace` object with FEASIBLE, UNLIKELY,



and ELIMINATED lists plus per-cause evidence strings. Also outputs a complexity hint: SIMPLE (one feasible cause), MODERATE (two), or COMPLEX (three or more). The agent uses this hint to decide whether to call the memory tool – for SIMPLE cases, memory retrieval is skipped entirely.

- **Retrieve Memory:** Queries the vector memory store for similar past training cases. The tool is skipped for SIMPLE cases and used for COMPLEX cases where the LLM benefits from seeing verified reasoning traces for similar historical faults.
- **Build Context:** Calls Stage 4. Converts the feature vector and decision space into the annotated semantic context, including threshold comparisons, cause evidence blocks, and data quality notes.
- **Predict Cause:** Calls Stages 5 and 6. Sends context to the LLM, extracts the predicted label using the cascading extraction strategy, and applies post-processing corrections. If the agent's previous prediction was marked as CORRECTED (the LLM chose an eliminated cause), the orchestrator allows one retry pass with an explicit explanation of why the previous prediction was rejected.

Memory Store Design

The memory layer uses ChromaDB for vector storage with `nomic-embed-text` (768-dimensional embeddings) as the encoder, running locally via Ollama. It operates in two modes:

Training mode (read and write): After each training sample is processed, its feature embedding is stored in the vector store alongside the full LLM reasoning trace and the ground-truth label. Because the pipeline runs four inference passes per question, an intra-case evolution rule governs which version of the reasoning is kept:

- If the existing entry is wrong and the new one is correct, the new entry replaces it unconditionally.
- If both are correct and the new reasoning is longer (higher quality score), the new entry replaces it.
- If the existing entry is correct and the new one is wrong, the existing entry is kept – a correct memory is never downgraded.

Test mode (read-only): Memory is kept frozen during test-time inference. Retrieved cases provide reliable reasoning patterns from similar past faults, but test predictions are not allowed to write back into memory, so they cannot affect or contaminate the training data.

When memory retrieval is used, the top three most similar cases are added to the LLM's context. The model is given the feature summaries, the correct labels, and the full reasoning traces from those past cases instead of just the final answer.

Random Forest Baseline

In parallel with the agent work, we trained a Random Forest classifier directly on the 52-element feature vector, bypassing the LLM entirely for the classification step. The feature set includes the 11 raw diagnostic measurements from Stage 2, binary cause trigger flags, compound features, and data quality flags (indicating whether location, engineering, and neighbour data were available).

The classifier handles missing values by filling them with the median. It was evaluated using five-fold stratified cross-validation to ensure reliable performance across different data splits.

The result was better than the memory-augmented agent, confirming that the engineered feature set carries most of the diagnostic signal. The Random Forest (RF) model acts as a useful sanity check for the pipeline. If a simple, deterministic classifier using the same features achieves similar accuracy to the agent, it indicates that the LLM's main value lies in handling cases where the features alone are unclear or ambiguous.

Accuracy Progression

System	Qwen2.5-1.5B Accuracy
Baseline with simple prompts	12%
Original four-stage pipeline	65%
Redesigned seven-stage pipeline	72%
Memory-augmented tool-based agent	82%

Research Paper

The findings from our submission to the AI Telco Challenge are written up as a research paper. The central argument is that the effectiveness of LLM-based pipelines for technical diagnostic tasks depends almost entirely on what happens before the model sees the input. Specifically, whether domain-specific computation has been separated out, whether the decision space has been shaped with physical constraints, and whether the evidence has been presented in a form that matches what language models are good at processing. The paper covers the pipeline architecture, per-stage ablation results, the memory retrieval system and discussion on the improvement.

Summary

Month	Summary
January	Built a four-stage LLM inference pipeline for telecom fault diagnosis. Key technical choices: column alias registry for inconsistent headers, max-distance over mean-distance for geometry features, dual Elimination and Thorough reasoning modes, seven-pattern cascading answer extraction, four-run majority voting. Finished 23rd of 199 on the competition leaderboard. Qwen2.5-1.5B accuracy: 65%.
February	Planned to extend the Trace AI Agent with a three-level semantic cache (exact match, cosine similarity, query hash), a 21-type error taxonomy with 15 correction strategies (programmatic and LLM-based), and prompt refinement.
March	Redesigned the pipeline into a seven-stage architecture with an explicit deterministic computation stage, a decision-space shaping stage, and a semantic abstraction stage. Separated LLM reasoning from computation, added per-class accuracy feedback and automatic hints. Accuracy improved to 72%. Started writing the paper.
April	Converted pipeline stages to callable tools in a ReAct agent loop. This ensures the pipelines are not static and change dynamically. Added a ChromaDB-backed memory store with intra-case evolution rules and reliability-weighted retrieval. Accuracy improved to 72%. Trained a Random Forest baseline on the 52-element feature vector with domain-specific override rules. Got accuracy as 94.56%. Research paper finalized.